

Prueba de Programación

Física y Astronomía

Pregunta 1 – Ley de Enfriamiento de Newton (np.polyfit)

Contexto físico:

Un objeto caliente se enfría siguiendo aproximadamente una relación lineal entre su temperatura T y el tiempo t durante intervalos cortos. Un experimento registra la temperatura de una taza de café a lo largo del tiempo.

Modelo matemático:

$$T(t) = T_0 - \alpha t$$

donde: T_0 °C (temperatura inicial), α °C/min (tasa de enfriamiento) y σ_T °C (incertidumbre constante en las mediciones).

Datos experimentales:

Se tomaron mediciones cada 5 minutos durante 50 minutos:

$$t = [0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50] \text{ minutos}$$

Datos sintéticos proporcionados:

Los datos experimentales (con ruido) ya han sido generados y están disponibles en las variables **t_enfriamiento** y **T_obs** (T_observable).

Recomendación: Primero que todo use `plt.scatter(t_enfriamiento, T_obs)` para observar los datos.

Tareas a realizar (paso a paso):**1. Ajuste lineal con np.polyfit:**

- Utilice `np.polyfit(t, T_obs, deg=1)` para obtener los coeficientes del ajuste
- Extraiga la pendiente ($-\alpha_{\text{estimado}}$) y el intercepto ($T_{0,\text{estimado}}$)
- Calcule las temperaturas ajustadas: $T_{\text{estimado}} = T_{0,\text{estimado}} - \alpha_{\text{estimado}} \cdot t$

2. Cálculo del coeficiente de determinación R^2 :

- Calcule la suma de cuadrados de residuos: $SS_{\text{res}} = \sum (T_{\text{obs}} - T_{\text{estimado}})^2$
- Calcule la suma total de cuadrados: $SS_{\text{tot}} = \sum (T_{\text{obs}} - \bar{T}_{\text{obs}})^2$
- Compute $R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$
- Imprima el valor de R^2 con 4 decimales

3. Visualización de resultados:

- Grafique los datos observados (**T_obs**) como **scatter points**.
- Grafique la línea de ajuste T_{estimado} como línea continua.

4. Reporte de resultados:

- Imprima los parámetros estimados: T_0 y α con 3 decimales.
- Imprima una conclusión sobre la calidad del ajuste basándose en R^2 (bueno si $R^2 > 0,95$)

Pregunta 2 – Decaimiento Radiactivo (scipy.optimize.curve_fit)

Contexto físico:

Una muestra radiactiva decae exponencialmente con el tiempo. Se desea estimar la cantidad inicial de material y la constante de decaimiento a partir de mediciones experimentales.

Modelo matemático:

$$N(t) = N_0 e^{-\lambda t}$$

donde: N_0 es el número inicial de átomos radiactivos, λ es la constante de decaimiento, y $N(t)$ es el número de átomos restantes en el tiempo t . $\lambda = 0,15 \text{ min}^{-1}$ (constante de decaimiento)

Incertidumbre en las mediciones: $\sigma_N(t) = 20 + 0,02 \cdot N(t)$ (variable, combina error instrumental y estadístico)

Datos experimentales:

Se realizaron 12 mediciones en los siguientes tiempos:

$$t = [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22] \text{ minutos}$$

Datos sintéticos proporcionados:

Los datos experimentales (con ruido) y sus incertidumbres ya han sido generados y están disponibles en las variables `t_decaimiento`, `N_obs` y `sigma_N`.

Recomendación: Primero que todo use `plt.scatter(t_decaimiento, N_obs)` para observar los datos.

Tareas a realizar (paso a paso):

1. Definición del modelo para `curve_fit`:

- Defina una función Python `def modelo_decaimiento(t, N0, lambd)` que implemente la fórmula $N(t) = N_0 e^{-\lambda t}$ use `np.exp()`.

2. Ajuste no lineal con `curve_fit`:

- Proporcione valores iniciales razonables: `p0 = [4500, 0.1]` (subestimados intencionalmente)
- Realice el ajuste con ponderación: `curve_fit(modelo_decaimiento, t, N_obs, p0=p0, sigma=sigma_N, absolute_sigma=True)`
- Extraiga los parámetros óptimos `popt` y la matriz de covarianza `pcov`
- Calcule `N_fit` usando los parámetros ajustados

3. Extracción de incertidumbres:

- Calcule las incertidumbres de los parámetros: `perr = np.sqrt(np.diag(pcov))`
- Extraiga individualmente: `sigma_N0 = perr[0]` y `sigma_lambda = perr[1]`

4. Cálculo de χ^2 reducido:

- Calcule los residuos ponderados: $r_i = \frac{N_{\text{obs},i} - N_{\text{fit},i}}{\sigma_{N,i}}$
- Compute $\chi^2 = \sum r_i^2$
- Calcule grados de libertad: $\nu = n_{\text{datos}} - n_{\text{parámetros}} = 12 - 2 = 10$
- Obtenga $\chi_{\text{red}}^2 = \frac{\chi^2}{\nu}$
- Imprima χ^2 , ν y χ_{red}^2 con 4 decimales

5. Visualización con barras de error:

- Use `plt.errorbar(t, N_obs, yerr=sigma_N, fmt='o')` para los datos con barras de error
- Genere una curva suave: cree `t_fino = np.linspace(0, 22, 200)` y calcule `N_fino`
- Incluya leyenda con los parámetros ajustados (formato: $N_0 = \text{valor} \pm \text{error}$)

Pregunta 3 – Análisis de Complejidad del Algoritmo Bubble Sort

Contexto computacional:

El algoritmo Bubble Sort es uno de los algoritmos de ordenamiento más simples, pero también uno de los menos eficientes para listas grandes. Su complejidad teórica es $O(n^2)$, lo que significa que el número de operaciones crece cuadráticamente con el tamaño de la lista.

Algoritmo proporcionado:

```
def bubble_sort(lst):
    n = len(lst)
    for i in range(n-1):
        for j in range(n-1-i):
            if lst[j] > lst[j+1]:
                lst[j], lst[j+1] = lst[j+1], lst[j]
    return lst
```

Datos experimentales proporcionados:

Se han generado 10 listas desordenadas aleatoriamente:

- Lista_1 con $2^1 = 2$ elementos
- Lista_2 con $2^2 = 4$ elementos
- Lista_3 con $2^3 = 8$ elementos
- ⋮
- Lista_10 con $2^{10} = 1024$ elementos

Nota: Las listas ya están definidas en el código inicial proporcionado, no es necesario usar `.copy()` ya que las listas siempre vuelven a su estado al volver a correr el código.

Tareas a realizar (paso a paso):**1. Modificación del algoritmo para instrumentación:**

- Cree una nueva función `bubble_sort_instrumentado(lst)` basada en el algoritmo original
- Agregue dos contadores: `comparaciones = 0` e `intercambios = 0`
- Incremente `comparaciones` cada vez que se evalúe `if lst[j] > lst[j+1]`
- Incremente `intercambios` cada vez que se realice un intercambio
- Que retorne: `(lista_ordenada, comparaciones, intercambios)`

2. Análisis experimental de las 10 listas:

- Cree tres listas vacías para almacenar resultados: `longitud_lista = []`, `num_comparaciones = []`, `num_intercambios = []`
- Use un bucle `for i in range(1, 11)` para procesar cada lista `Lista_i` (recuerda el `.append()`).

3. Visualización de resultados gráfico:

- Graficar con `plt.plot(longitud_lista, num_comparaciones, 'o-', label='Comparaciones')`
- Graficar con `plt.plot(longitud_lista, num_intercambios, 's--', label='Intercambios')`
- Agregar etiquetas, leyenda y título